

HelixTranslate

HelixTranslate

検証済みモデルによる書籍翻訳——設計段階から誠実さを貫き、決して静かなフォールバックを許さない

概要

HelixTranslateは、Goベースの高性能電子書籍翻訳プラットフォームです。100以上の言語間で書籍を翻訳する際、検証済みのLLMプロバイダーを使用し、リアルタイムのWebSocket監視と「静かなフォールバックを許さない」厳格なルーティングポリシーを採用。品質低下を密かに許容するのではなく、明確なエラーとして失敗します。

短い説明

Goベースの汎用電子書籍翻訳ツールキット。FB2、EPUB、TXT、HTML、PDF、DOCXに対応し、100以上の言語間で翻訳を実行。最強の検証済みLLM（LLMsVerifierブリッジ経由）を使用し、REST/HTTP-3およびgRPC API、分散処理、リアルタイムWebSocket監視ダッシュボードを備えています。

長い説明

HelixTranslateは、企業向けのGoベースシステムで、LLMプロバイダーを用いて書籍全体を言語間で翻訳します。対象は段落や断片ではなく、書籍全体を端から端まで翻訳することに特化しています。複数の電子書籍フォーマット（FB2、EPUB、TXT、HTML、PDF、DOCX）の解析・再生成が可能で、100以上の言語に対応し、自動言語検出機能を備えています。また、CLIツールとAPIサーバー（HTTP/3上のREST、gRPC、およびWebSocketイベントストリーム）を提供し、ターミナルワークフローにもサービスメッシュにも柔軟に組み込めます。

このシステムの最大の特徴は、モデルの選択方法にあります。特定のプロバイダーをハードコーディングしてその健全性を期待するのではなく、HelixTranslateはすべてのモデル権限をLLMsVerifierブリッジ（pkg/

bridge) に委ねます。ブリッジは最も強力な検証済みのAPIモデルを選択し、スコア順に決定的なフォールバックチェーンを返します。モデルの適格性は、応答性、コード品質、機能の豊富さ、信頼性を総合的に評価した加重スコアによって決定されるため、翻訳を担当するモデルは設定ファイルに記載されているからではなく、実際に機能することを証明したからこそ選ばれるのです。

さらに重要なのは、システムがコードレベルで「静かなフォールバック禁止」を徹底している点です。プロバイダーのAPIキーが存在しない場合や、利用不可能なプロバイダーを明示的に要求した場合、パイプラインは密かにプロバイダーを切り替えたり、ローカルランタイムにフォールバックして「問題なし」と偽るのではなく、明確なハードエラーを返します。このルールは専用のビルド前ゲートとペアのミュートーションテストによって厳格に管理されています。ローカルランタイム (Ollama、llama.cpp) は意図的にデフォルトパスから除外されており、検証済みモデルの代わりに弱いエンジンが密かに代用されることは決してありません。

翻訳コアの周囲には、リアルタイムのWebSocket監視サブシステムが配置されています。翻訳CLIは型付きイベントを監視サーバーに送信し、ライブウェブダッシュボードを駆動します。また、リモートのSSHワーカーがワークロードを分散処理します。さらに、一貫性を高める多段階ポリッシング、事前品質分析、長文入力時のコスト抑制を目的とした翻訳キャッシュ、視覚駆動の品質保証が重層的に実装されています。このプラットフォーム全体は、「ハッターリ禁止」のエンジニアリング憲章に基づいて設計されており、テストは単なるグリーンチェックマークではなく、実際のユーザー体験を証明するものでなければなりません。そのため、ミュートーションテストが必須とされています。

コンテンツ

なぜ開発したのか

長編書籍を信頼性高く、そして誠実に翻訳するため——「劣化はしているが一応存在する」翻訳を決して届けない。設計の前提は、欠落した翻訳や検証不可能な翻訳は、大きく明確なエラーとして扱われなければならないというものであり、モデルの選択は常に、ハードコードされた推測や静かなローカルフォールバックではなく、真に検証されたプロバイダーに収束する必要がある。

なぜゲームチェンジャーなのか

ほとんどのLLM翻訳パイプラインは静かに失敗する——弱いモデルにこっそりフォールバックしたり、ローカルランタイムに切り替わったり、テストスイートが緑色のまま部分的な出力を吐き出しながら、誰も品質の崖を察知しない。HelixTranslateは、この失敗モードそのものを構造的に不可能にする。モデルの選択は検証によって制御され、フォールバックの連鎖は決定論的かつ完全に透明であり、「キーがない／検証済みモデルがない」という状況は、静かな肩をすくめるような対応ではなく、正直なハードエラーとして処理される。このたった一つの設計判断により、「この翻

訳は本当に、能力があり検証されたモデルで実行されたのか？」という問いは、確認のしようがない希望から、システムが代わりに保証するものへと変わる。

革新的な点

- 検証ゲート型モデルルーティング LLMsVerifierブリッジを介して、最も強力な検証済みモデルが自動選択される。これにより、オペレーターはベンダー名ではなく意図を宣言し、ダウンしている可能性のあるプロバイダーを手動で選ぶことはない。
- ノー・サイレント・フォールバック保証のコードによる強制 4つの明示的なルーティング経路（モック／明示的検証器／明示的プロバイダー／ブリッジデフォルト）があり、それぞれが静かに切り替わるのではなくハードエラーを発生させる。加えて、デフォルトパスからローカルランタイムを意図的に排除し、フォールバック先となる弱い選択肢を根絶。
- 機械的な強制 CM-NO-LOCAL-RUNTIMEプリビルドゲートとペアのミュートーションテストにより、ビルド時にデフォルトパス上でローカルランタイムクライアントが構築されないことを保証。この保証は、もし破られればビルドが失敗するため、劣化することはない。
- 決定論的かつスコア順のフォールバック連鎖 検証済みモデル間でのプロバイダー間フェイルオーバーは許可され、完全に透明。これは、禁止されているサイレントフォールバックとは原則的に区別される。どの能力あるモデルが作業を受け持ったかが常に明らか。
- リアルタイム**WebSocket**モニタリング 型付き翻訳イベントがダッシュボードにライブストリーミングされ、分散SSHワーカーにより、書籍長のジョブも可視化され並列処理される。ブラックボックスにはならない。
- アンチブラフテスト体制 ミュートーションテスト、否定的アサーション、実システムでの実行、ビジョン駆動のQAを組み合わせ、「テストが通る」ことが「機能が実際に動作する」ことを静かに隠蔽することがないように保証。

最大の技術的課題とその解決方法

- 誠実な翻訳パイプラインの保証（サイレントな劣化なし）。 LLMsVerifierブリッジにモデルの権限を集中させ、単一の決定ポイントで管理することで解決。4つの明示的なルーティングブランチをコード化し、それぞれが推測ではなく大きなエラーを発生させる。デフォルトパスからローカルランタイムのフォールバックを完全に排除し、ビルドゲートとミュートーションテストでルールを固定。保証が破られればビルドが失敗する。
- 「テストは通るが機能は壊れている」。この失敗モードを憲章で明示し、アンチブラフテスト体制で克服。実装の細部ではなく、ユーザーに見える具体的なアサーションを採用。モックはユニットテストに限定し、実システムをループに組み込む。ミュートーションテストを必須とし（機能を意図的に壊してもテストが必ず失敗する）、ビジョン検証QAで実際の出力を確認。

- **長編・マルチフォーマットの品質。** 書籍長の入力は一貫性とコストの両面で負荷が大きい。マルチパスのポリッシングでテキストを再検討し、事前分析フェーズでジョブの規模を把握。翻訳キャッシュにより、同じ文章に二度コストをかけない。

コンテンツ

テックスタック

- **Go** — 並行処理プリミティブが、複数の章を同時にパース・翻訳・ストリーミングする処理に自然にマッピングされるため採用。高並行性バックエンドとして、`digital.vasic.translator`モジュールを使用。
- **Gin** — 高速かつ最小限のHTTPルーターとして、REST APIのサーフェスを提供するために採用。
- **QUIC / HTTP/3 (quic-go)** — REST APIに、低遅延で現代的なトランスポートを提供し、不安定なネットワーク環境でも安定動作を実現するために採用。
- **gRPC + Protocol Buffers** — 型安全性と高パフォーマンスを両立したサービスインターフェースとして、プログラムからの呼び出しに対応するRESTと併せて採用。
- **Gorilla WebSocket** — リアルタイムの型付き翻訳イベントストリームを監視ダッシュボードにライブフィードするために採用。
- **PostgreSQL, SQLite, Redis** — 意図的な3層構造: PostgreSQLは耐久性のあるリレーショナルデータ用、SQLiteは組み込み／ローカル状態用（ブリッジの検証済みモデルストア`data/verified_models.db`も担当）、Redisはホットキャッシュとして機能。
- **unidoc/unioffice + unipdf** — 難易度の高いフォーマット（DOCXおよびPDF）のパースと再生成を担当し、マルチフォーマットの電子書籍を忠実に往復変換するために採用。
- **Cobra** — `unified-translator`およびその関連ツールを駆動するCLIフレームワークとして採用。
- **golang-jwt (JWT HS256)** — ステートレスなAPI認証のために採用。IPごとのトークンバケットによるレート制限と、TLS／QUICトランスポートセキュリティを組み合わせ、サーフェスの堅牢性を確保。
- **LLMsVerifierブリッジ (pkg/bridge)** — 中核要素: 最も信頼性の高い検証済みモデルと、その決定論的フォールバックチェーンを提供し、サイレントフォールバック禁止の保証を一元的に強制する役割を担う。
- **Testify** — Goのテストスイートに採用。専用の`provider_routing_test.go`や、正直性ルールを厳守するためのミュレーションゲートも含む。
- **Docker / Podman (ルートレス) + Compose** — コンテナ化された分散デプロイメント (`docker-compose.distributed.yml`) のために採用。セキュリティ強化のため、ルートレスPodmanを使用。

ステータスおよび正直性に関する注意事項

- ステータス：ベータ版。機能は実装済みだが、VERSION／Makefile／AGENTS.md間でバージョン表記に不整合があるため、現状は未確定とみなす。
- ライセンス：未定。READMEではMITと記載されているが、LICENSEファイルとの整合性は未確認。明記する前に検証が必要。
- ダッシュボード／モニターエンドポイントはローカルホスト専用で公開されていない。ドキュメントに記載されたWebSocketのパフォーマンス数値は目標値であり、検証済みではない。ARCHITECTURE.mdには削除済みのOllama／ローカルエンジンが記載されたまま（情報が古い）。

優先度レベル： Helixプライマリ（LLMインフラストラクチャクラスター）。Helixプラットフォームファミリー内では、HelixTrackに次ぐ位置づけ。